

Luma - Project Defense & Codebase Walkthrough

This document is your ultimate cheat sheet for presenting and defending your codebase. It connects every major feature directly to the specific files and functions where it is implemented.

1. High-Level Summary (What the App Does)

Luma is an AI-powered Loan Data Verification Copilot. It allows financial teams to upload raw, messy loan data files (CSVs). The system automatically runs validation rules against the data and flags any errors (Exceptions) to a queue. Reviewers use an AI Copilot (powered by Google Gemini) to explain why the data failed, suggest mathematical corrections, classify severity, and draft audit notes. Once clean, the data can be exported to downstream banking systems.

2. Feature-to-Code Mapping

If the interviewer asks: "Show me where [Feature] is implemented..."

A. The AI Copilot Features (The "Brain")

All AI business logic is centralized in a single service layer. File:

`apps/api/src/services/ai.service.ts`

- "Explain why this failed & suggest a fix": Go to `explainException()`
- "Summarize this entire upload batch": Go to `summarizeBatch()`
- "Re-evaluate the severity of this error": Go to `classifySeverity()`
- "Translate my natural language into a validation rule": Go to `suggestRule()`
- "Draft an audit note for this correction": Go to `draftReviewerNote()`

AI Configuration & SDK Integration: File:

`apps/api/src/lib/ai.ts`

- This is where the Vercel AI SDK is configured to connect to Google Gemini (`getModel()`).

B. The API & Frontend Communication (The "Bridge")

How does the React frontend talk to the Express backend? File:

`apps/web/src/lib/api.ts`

- **What it does:** Uses `axios` to create typed REST endpoints.
- **Key objects:** `uploadsApi` , `loansApi` , `exceptionsApi` , `aiApi` .
- **Frontend caching:** React components use `@tanstack/react-query` to call these Axios functions seamlessly with loading states.

C. Database & Schema (The "Source of Truth")

How are users, loans, and exceptions stored? **File:**

`apps/api/prisma/schema.prisma`

- **Users/Auth:** Managed by `better-auth` (User, Session, Account tables).
- **Uploads:** `UploadBatch` tracks the status of a CSV import.
- **Loans:** `Loan` (raw incoming data) and `VerifiedLoan` (cleaned output data).
- **Errors:** `Exception` tracks the specific validation failures.
- **Auditing:** `AuditLog` permanently tracks who did what.

Database Connection: File: `apps/api/src/lib/prisma.ts`

- Initializes the Prisma Client to query the PostgreSQL database.

D. Authentication & Security (RBAC)

How do you handle login and Roles (Operator, Reviewer,

Consumer)? **Backend Auth Config:** `apps/api/src/lib/auth.ts`

- Configures `better-auth`, handling the session cookies and user roles. **Seed Data (Default Users):**
`apps/api/src/seed.ts`
- Shows the exact setup for the `data_operator`, `reviewer`, and `data_consumer` roles.

E. The Monorepo Structure

If asked about how the project is organized:

- `apps/api/`: The Express.js backend.
- `apps/web/`: The React/Vite frontend.
- `packages/types/`: Shared TypeScript types (and Zod schemas) ensuring the frontend and backend agree on the data shapes.
- `docker-compose.local.yml` / `docker-compose.prod.yml`: Docker configuration for running the full stack securely and

reproducibly.

3. Potential Interview Questions & How to Answer Them

Q1: "How do you ensure the AI returns structured data instead of just raw text?"

Answer: "I use the Vercel AI SDK's `generateObject` function in combination with **Zod** schemas. If you look at `apps/api/src/services/ai.service.ts`, you'll see I define a strict Zod schema for the expected response (e.g., `confidence`, `suggestedValue`), and the AI is forced to return JSON that perfectly matches that schema."

Q2: "What happens if the AI fails or the API key is missing?"

Answer: "I built a fallback mechanism. In `apps/api/src/Lib/ai.ts`, there is an `isMockAi` toggle. If true, the system returns deterministic, hardcoded mock recommendations so development and testing can continue without hitting rate limits or needing an active internet connection."

Q3: "How is this application deployed?"

Answer: "It's fully containerized. If you look at `docker-compose.prod.yml`, it builds separate containers for the API and the Web frontend, spins up a PostgreSQL database, runs Prisma migrations automatically, and uses a custom

internal Docker network to ensure the database is never exposed to the public internet."

Q4: "How do you share types between the frontend and backend?"

Answer: *"Because it's a Turborepo monorepo, I created a `packages/types` folder. Both `apps/api` and `apps/web` import their interfaces from this central package. This means if I change the shape of an API response on the backend, the frontend TypeScript compiler will immediately throw an error if it's not updated."*